
Introduction to Database

XXXXXX

Project

Deadline: Monday 09/08/2026 @ 23:59

[Total Mark is 20]

Student Details:

CRN:50246

Name: Dana Mohammed Alajmi

ID: 240040990

Name: Amani Barjas

ID: 240047827

Name: Dareen Alanazi

ID: 240022387

Name: Manar Mutaib Al- Harbi

ID: 240061219

Instructions:

- You must submit two separate copies (**one Word file and one PDF file**) using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, **convert text into image** or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You **MUST** show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Project Instructions

- You can work on this project as a group (minimum 3 and maximum 4 students). Each group member must submit the project individually with all group member names mentioned on the cover page.
- This project **worth 20 marks** and will be distributed as in the following:
 - Database Design. (6 marks)
 - Database Implementation. (6 marks)
 - SQL Operations. (6 marks)
 - Project Reflection. (2 marks)
- Each leader in the group must submit one report about their chosen Project via the Blackboard (**Email submission will not be accepted and will be awarded ZERO marks**) containing the following:
 - A. Task 1 : database design
 - B. Task 2: database implementation
 - C. All requested queries/results
 - D. Screenshots from MySQL (or any other software you use) of **all the tables** after population and **query results**.
 - E. Reflection (300–500 words) describing the team's experience, challenges, design decisions, and AI usage (if applicable).
- Submit one **SQL file** containing the complete database implementation, including database creation, tables, data insertion, required queries, VIEW, and TRIGGER.
- Submit a **GitHub** (or GitLab) repository containing the complete project with regular commits and meaningful commit messages.
- Submit a **progress report** describing the completed work, key decisions, challenges, future plan, and contribution of each group member.
- Include **links** to the live report document, GitHub/GitLab repository, and Google Colab notebook (if applicable) in the **Project Artifacts** section of the final report.
- You are advised to make your work clear and well presented; marks may be reduced for poor presentation. This includes filling in your information on the cover page.
- You MUST show all your work, and text **must not** be converted into an image unless specified otherwise by the question.
- **Late submission will result in ZERO marks being awarded.**
- The work should be your own. Copying **from students or other resources, including AI software, will result in ZERO marks.**

*Learning**Outcome(s):**CLO1, CLO2, CLO3,**CLO4, CLO5*

Project Description

Design and Implementation of a Smart Clinic Database System

A private clinic currently manages its patient information manually, making it difficult to organize appointments, treatments, and payments. The clinic has decided to develop a database system to improve data organization and retrieval.

Your team is required to design, implement, and test a complete relational database using the concepts learned throughout this course. The project should demonstrate your understanding of database design, ER/EER modeling, relational mapping, SQL implementation, and advanced SQL queries. The project must be completed using MySQL.

Task 1 – Database Design (6 Marks)

- Design a complete database for the clinic that includes at least six entities such as Patients, Doctors, Appointments, Treatments, Medicines, Payments, or any other appropriate entities.
- A complete ER/EER diagram.
- Primary and foreign keys.
- Appropriate relationships with cardinality.
- At least one specialization/generalization (EER feature).
- Any assumptions made during the design process.

Task 2 – Database Implementation (6 Marks)

Complete the following tasks using MySQL. Include screenshots of both your SQL code and the corresponding output (results) for each task:

- Create the database and all required tables.
- Use appropriate data types and constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, etc.).
- Insert at least 5 records into each main table.

Task 3 – SQL Operations (6 Marks)

Complete the following tasks using MySQL. For each task, include screenshots of both your SQL code and the corresponding output (results), along with a brief description (one to two sentences) explaining the purpose of the query and the information it is intended to retrieve.

- Write SELECT statements.
- Write JOIN queries.
- Write nested queries.
- Use aggregate functions with GROUP BY.
- Write UPDATE and DELETE statements.
- Create one VIEW.
- Create one TRIGGER.

Task 4 – Reflection (2 Marks)

Write a **300–500-word reflection** on your experience while completing this project by answering the following questions. Your responses should be clear, concise, and supported with specific examples from your own work.

1. Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.
2. Explain why you chose your specific approach/method/model over at least one alternative you considered, and what tradeoffs informed that decision.
3. If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.
4. What would you do differently if you were to redo this project with more time or resources?

Deliverables

1. **Project Report (PDF)** – System description, ER/EER diagram, relational schema, SQL code, screenshots, and reflection.
2. **SQL Script (.sql)** – Complete SQL implementation of the project.
3. **GitHub Repository** – Repository link with the complete project and commit history.
4. **Mid-Project Progress Report** – Summary of progress, challenges, decisions, and team contributions.
5. **Project Artifacts**
 - Live report document (with edit history): [link]
 - GitHub/GitLab repository (with commit history): [link]
 - Google Colab notebook (if applicable): [link]

[Project Solutions]

Table of Contents

Project Instructions	1
Project Description	2
Task 1 – Database Design (6 Marks)	3
Task 2 – Database Implementation (6 Marks)	4
Task 3 – SQL Operations (6 Marks)	5
Task 4 – Reflection (2 Marks)	6
Deliverables	7
Table of Contents	8
Project Overview	10
Background	10
Project Objective and Scope	10
Team Task Distribution	10
1. Task 1 - Database Design	11
1.1 Entity Catalogue	11
1.2 EER Specialization/Generalization	12
1.3 Relationships and Cardinalities	12
1.4 Relational Schema	13
1.5 Complete ER/EER Diagram	14
1.6 Design Assumptions	15
2. Task 2 - Database Implementation	16
2.1 Implementation Environment	16
2.2 Data Types and Integrity Constraints	16
2.3 Sample Data Population	17
2.4 Execution Sequence	17
2.5 MySQL Workbench Evidence	18

3. Task 3 - SQL Operations	22
3.1 Verified Result Summary	22
3.2 SELECT Statements	23
3.3 JOIN Query - Appointment Details	24
3.4 JOIN Query - Patient Prescriptions	25
3.5 Nested Query	26
3.6 Aggregate Functions with GROUP BY	27
3.7 UPDATE Statement	28
3.8 DELETE Statement and Rollback Verification	28
3.9 VIEW Creation and Result	30
4.10 TRIGGER Creation and Stock-Control Test	32
4. Task 4 – Project Reflection	35
5. Project Artifacts	37
.6 Appendix A - Mid-Project Progress Report	38
7. Appendix B - Complete SQL Implementation	43
8. Appendix C - Screenshot Evidence Index	58

Project Overview

Background

A private clinic that manages patient information manually can face difficulty organizing appointments, treatments, medicines, and payments. The Smart Clinic Database System replaces fragmented records with a structured relational database that centralizes the clinic's core information and preserves the relationships between clinical and financial activities.

Project Objective and Scope

The project designs, implements, and tests a complete MySQL database using ER/EER modeling, relational mapping, integrity constraints, fictional Saudi-context data, retrieval and reporting queries, controlled data changes, a reusable view, and an inventory-protection trigger.

Team Task Distribution

Student	ID	Primary Assignment	Primary Evidence
Dana. Alajmi	240040990	Group Leader and Repository Manager	Integrated report, evidence register, package structure, and repository preparation.
Amani Barjas	240047827	Task 1: Database Design	ER/EER diagram, entity catalogue, relationships, cardinalities, assumptions, and relational schema.
Dareen Alanazi	240022387	Task 2: Database Implementation	MySQL table definitions, constraints, sample data, and eight populated-table screenshots.
Manar Al-Harbi	240061219	Task 3: SQL Operations	SQL operations, purpose statements, and twelve matching MySQL Workbench evidence images.
All members	_____	Task 4: Project reflection	Project Reflection and final technical/document review.

1. Task 1 - Database Design

1.1 Entity Catalogue

The design contains eight connected relations. PERSON stores shared identity data, while PATIENT and DOCTOR extend it through an EER specialization. Transactional tables then represent appointments, treatments, prescribed medicines, and payments.

Entity	Primary Key	Purpose
PERSON	person_id	Supertype for shared personal and contact details.
PATIENT	patient_id	Patient-specific registration, blood type, insurance, and emergency contact data.
DOCTOR	doctor_id	Professional license, specialty, room, fee, and hire date.
APPOINTMENT	appointment_id	Scheduled interaction linking one patient with one doctor.
TREATMENT	treatment_id	Diagnosis, clinical notes, date, and treatment cost for an appointment.
MEDICINE	medicine_id	Medicine catalogue, strength, dosage form, price, and stock.
TREATMENT_MEDICINE	treatment_id + medicine_id	Associative relation storing prescription details.
PAYMENT	payment_id	Financial transactions associated with appointments.

1.2 EER Specialization/Generalization

PERSON is modeled as the supertype because patients and doctors share national identification, full name, birth date, gender, phone, email, and address attributes. PATIENT and DOCTOR are disjoint subtypes whose primary keys also act as foreign keys referencing PERSON. The specialization is documented as total and disjoint: every stored clinic person belongs to one subtype, and the same person is not assigned to both roles.

1.3 Relationships and Cardinalities

Relationship	Cardinality	Business Rule
PERSON - PATIENT	1 : 0..1	A person may appear once as a patient subtype.
PERSON - DOCTOR	1 : 0..1	A person may appear once as a doctor subtype.
PATIENT - APPOINTMENT	1 : 0..N	A patient may book many appointments; each appointment has one patient.
DOCTOR - APPOINTMENT	1 : 0..N	A doctor may attend many appointments; each appointment has one doctor.
APPOINTMENT - TREATMENT	1 : 0..1	An appointment may produce at most one treatment.
APPOINTMENT - PAYMENT	1 : 0..N	An appointment may have several payment transactions.
TREATMENT - MEDICINE	M : N	TREATMENT_MEDICINE resolves the many-to-many relationship.

1.4 Relational Schema

Relation	Relational Schema Notation
PERSON	PERSON(person_id PK, national_id UQ, full_name, date_of_birth, gender, phone UQ, email UQ, address)
PATIENT	PATIENT(patient_id PK/FK -> PERSON.person_id, blood_type, insurance_provider, emergency_contact_name, emergency_contact_phone, registration_date)
DOCTOR	DOCTOR(doctor_id PK/FK -> PERSON.person_id, license_number UQ, specialty, room_number UQ, consultation_fee, hire_date)
APPOINTMENT	APPOINTMENT(appointment_id PK, patient_id FK -> PATIENT.patient_id, doctor_id FK -> DOCTOR.doctor_id, scheduled_at, appointment_duration_minutes, status, reason, created_at)
TREATMENT	TREATMENT(treatment_id PK, appointment_id FK/UQ -> APPOINTMENT.appointment_id, treatment_date, diagnosis, treatment_notes, treatment_cost)
MEDICINE	MEDICINE(medicine_id PK, medicine_name + strength UQ, dosage_form, unit_price, stock_quantity, prescription_required)
TREATMENT_MEDICINE	TREATMENT_MEDICINE(treatment_id PK/FK -> TREATMENT.treatment_id, medicine_id PK/FK -> MEDICINE.medicine_id, dosage, frequency, duration_days, quantity, instructions)
PAYMENT	PAYMENT(payment_id PK, appointment_id FK -> APPOINTMENT.appointment_id, amount, payment_date, payment_method, payment_status, reference_no UQ)

1.5 Complete ER/EER Diagram

Figure 1 presents the complete model, including the PERSON specialization, entity attributes, keys, relationship labels, and cardinalities used in the relational implementation.

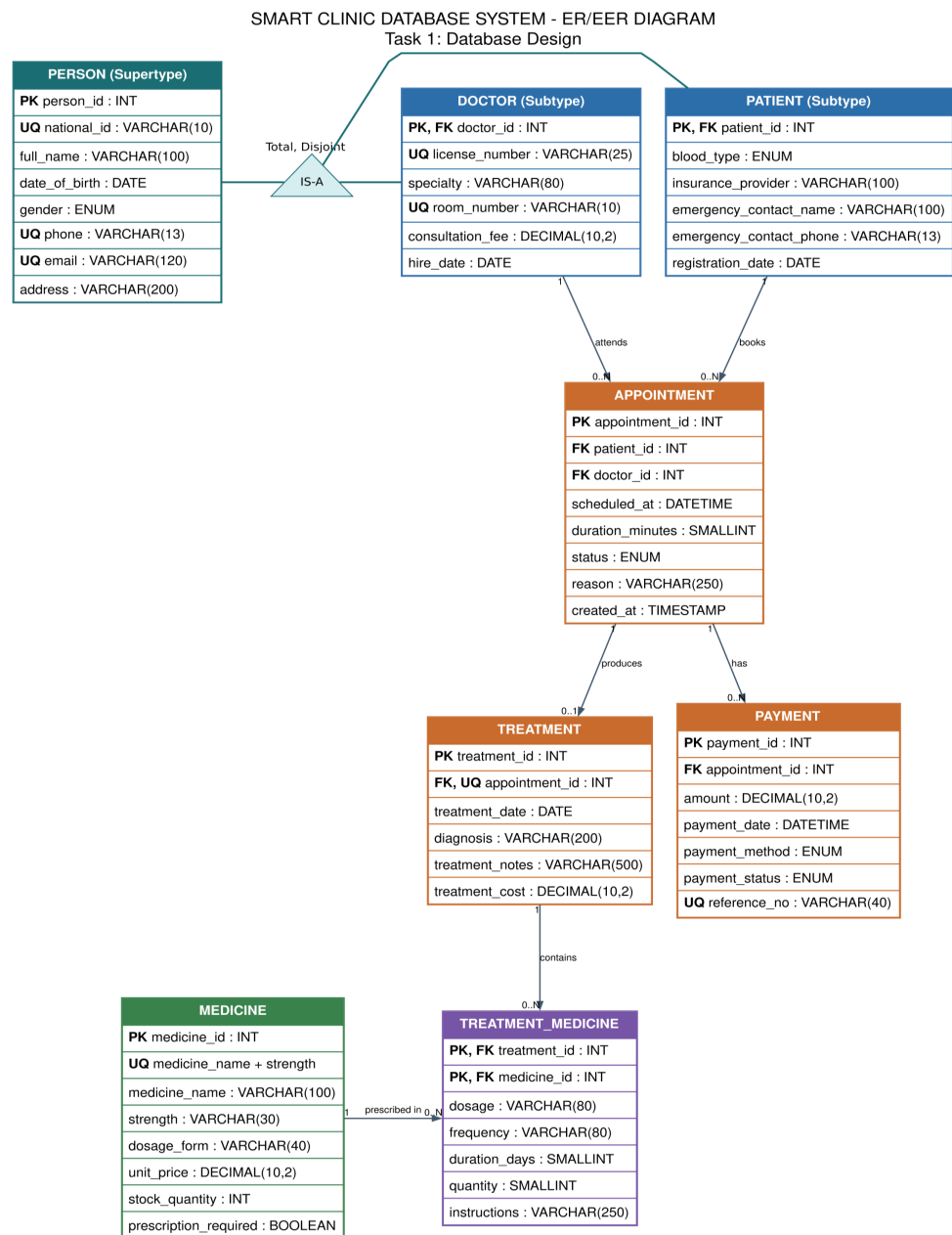


Figure 1. Complete ER/EER diagram for the Smart Clinic Database System.

1.6 Design Assumptions

- All monetary values are recorded in Saudi riyals (SAR).
- All names, identifiers, contact data, diagnoses, and payments are fictional and used only for academic work.
- A clinic person is assigned to exactly one subtype: PATIENT or DOCTOR.
- A doctor and a patient cannot each hold two appointments at the same scheduled date and time.
- An appointment may generate no more than one treatment record.
- A treatment can include several medicines, and a medicine can be used in several treatments.
- An appointment may have multiple payment transactions to support installments or insurance processing.
- The sum of payments is not forced to equal treatment and consultation charges because insurance adjustments may apply.
- Deletion is restricted for historical clinic transactions, while treatment-medicine rows follow their parent treatment.

2. Task 2 - Database Implementation

2.1 Implementation Environment

Item	Implementation
Database management system	MySQL 8.0 or newer
Development interface	MySQL Workbench
Database name	smart_clinic_db
Storage engine	InnoDB
Character set	utf8mb4
Collation	utf8mb4_0900_ai_ci
Implementation file	database/smart_clinic_database.sql

2.2 Data Types and Integrity Constraints

Constraint	Purpose in the Database
PRIMARY KEY	Uniquely identifies every entity and transaction.
FOREIGN KEY	Preserves valid references between related tables.
NOT NULL	Requires essential personal, clinical, and financial values.
UNIQUE	Prevents repeated national IDs, phones, licenses, rooms, payment references, and appointment slots.
CHECK	Rejects negative amounts or stock, invalid durations, and incorrectly formatted Saudi identifiers.
ENUM	Limits gender, blood type, appointment status, payment method, and payment status to approved values.

2.3 Sample Data Population

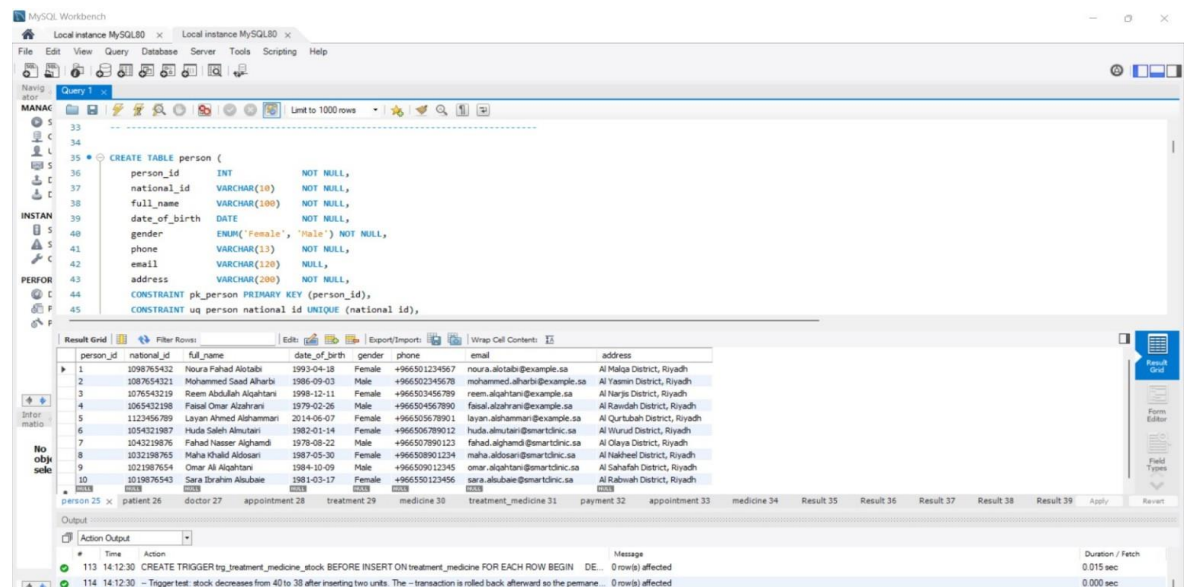
Table	Rows	Population Summary
PERSON	10	Five patients and five doctors share the supertype.
PATIENT	5	Five fictional clinic patients.
DOCTOR	5	Five doctors representing different specialties.
APPOINTMENT	5	Five completed appointments.
TREATMENT	5	One treatment for each completed appointment.
MEDICINE	5	Five medicines with stock and price data.
TREATMENT_M EDICINE	7	Seven prescription-detail combinations.
PAYMENT	5	Five completed payment records.

2.4 Execution Sequence

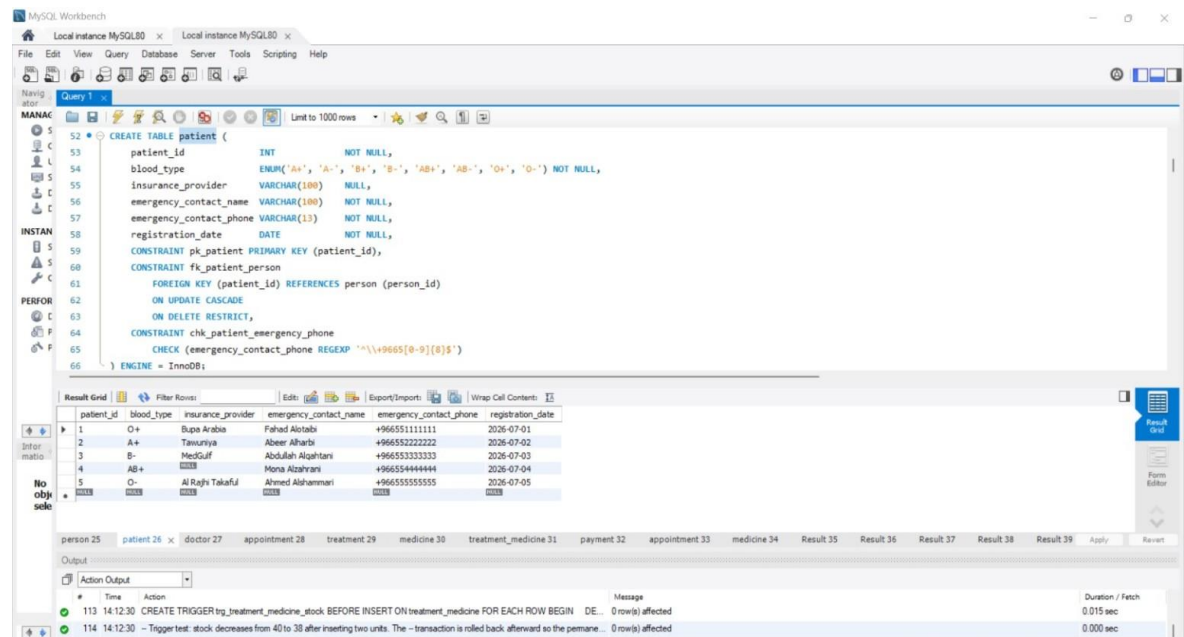
1. Drop and recreate smart_clinic_db so the script can be tested repeatedly in a clean environment.
2. Create PERSON first, followed by the PATIENT and DOCTOR subtypes.
3. Create APPOINTMENT, TREATMENT, MEDICINE, TREATMENT_MEDICINE, and PAYMENT in foreign-key dependency order.
4. Insert the fictional records inside a transaction, using the same parent-to-child dependency order.
5. Commit the sample data and run the table-display statements used for MySQL Workbench evidence.

2.5 MySQL Workbench Evidence

The following screenshots were captured from the completed MySQL Workbench execution. Together they demonstrate all eight populated tables and the SQL definitions associated with the implementation.

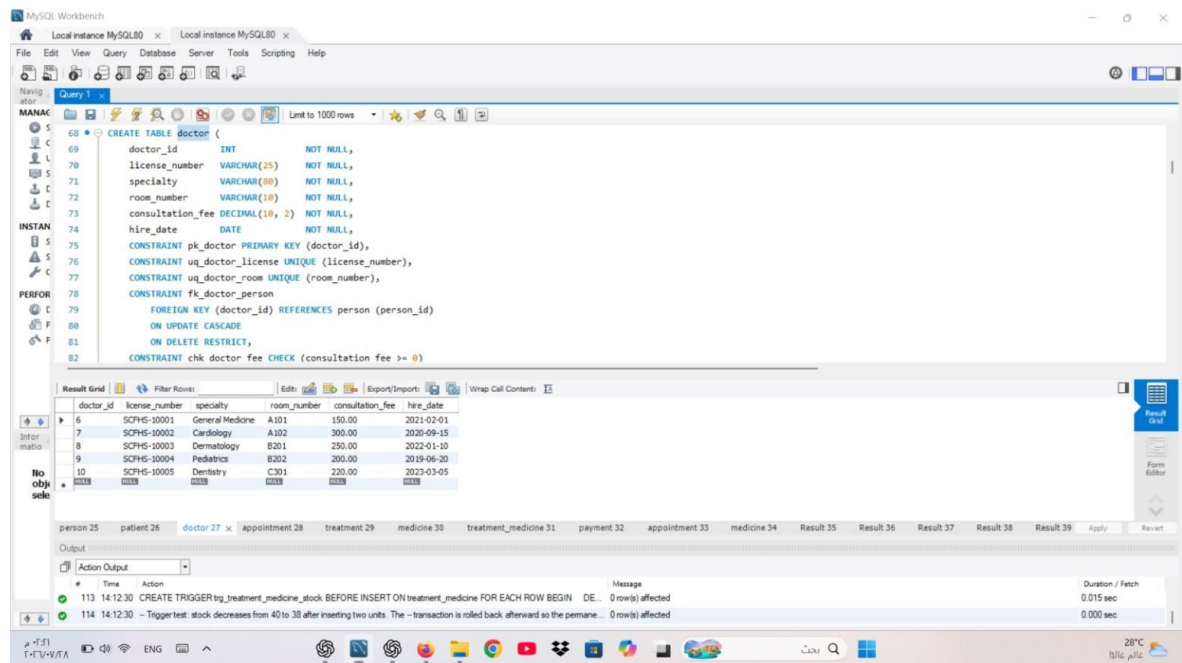


Screenshot 2.1. PERSON table definition and ten populated person records.

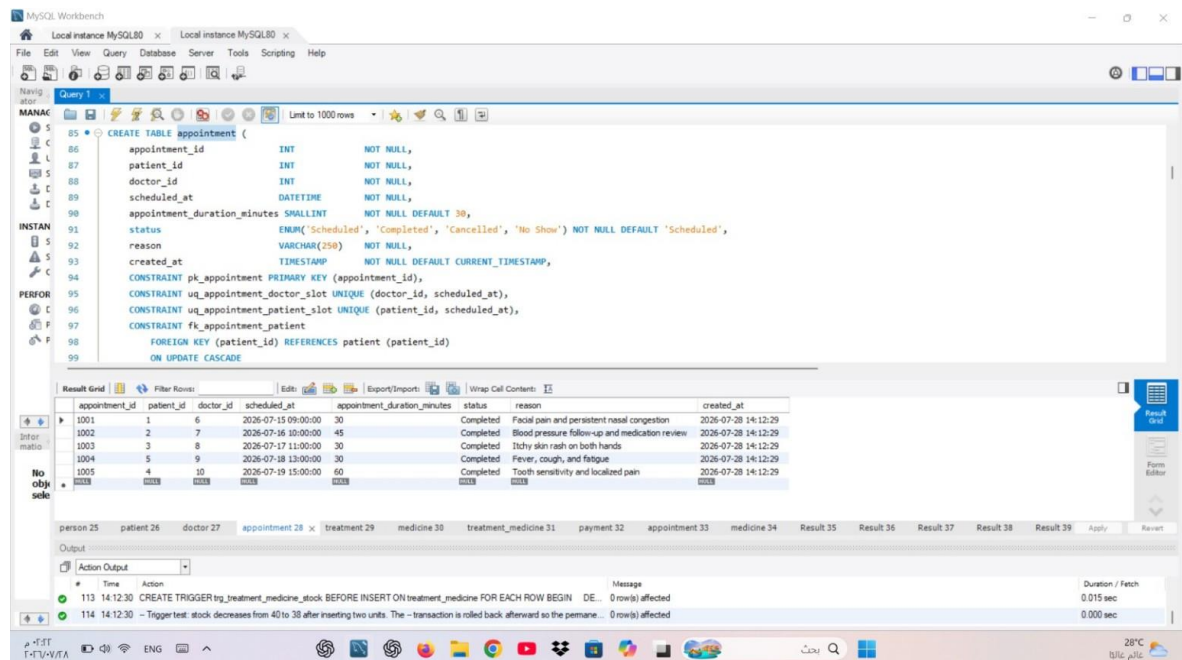


Screenshot 2.2. PATIENT subtype definition and five populated patient records.

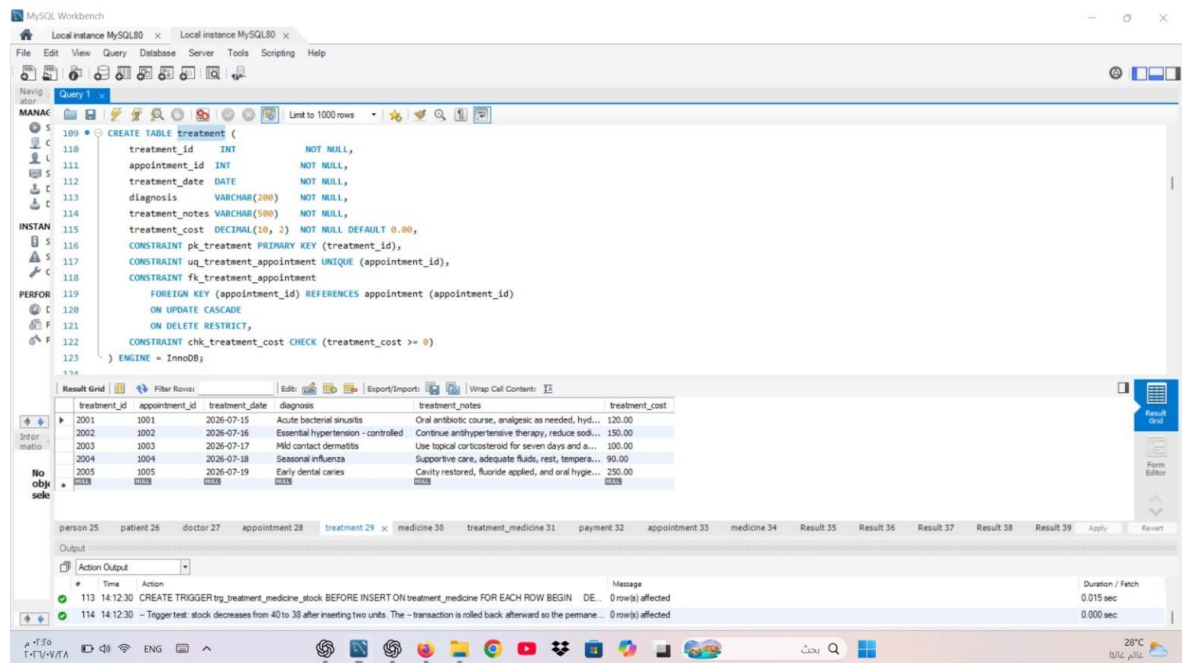
Task 2 - Database Implementation



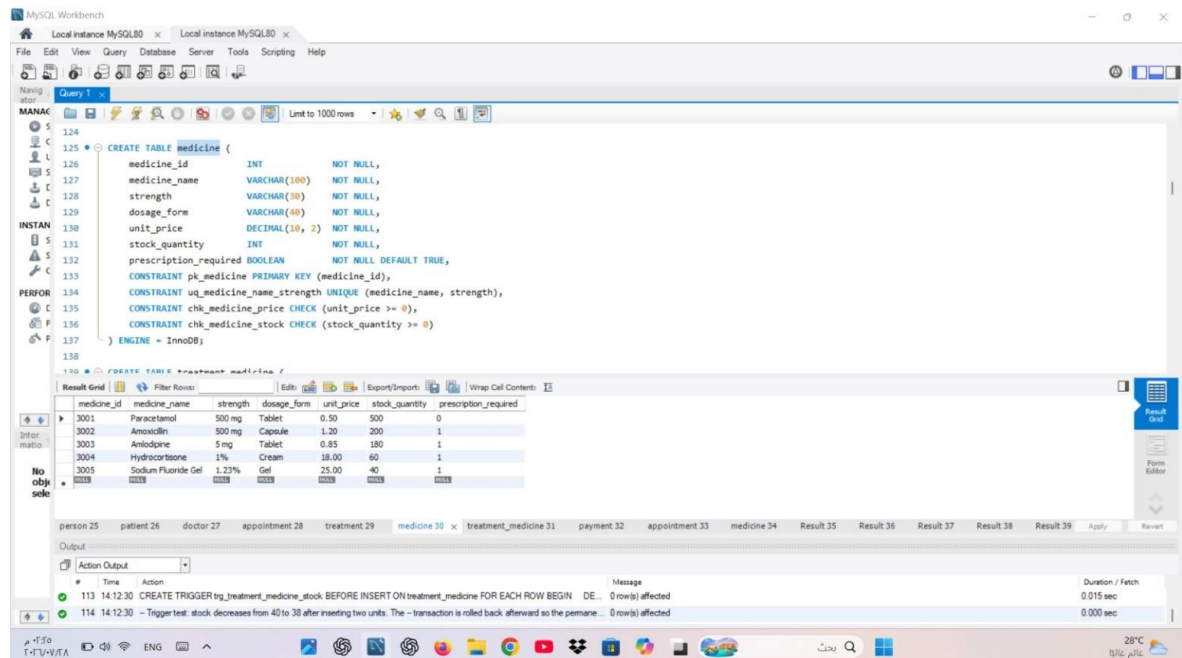
Screenshot 2.3. DOCTOR subtype definition and five populated doctor records.



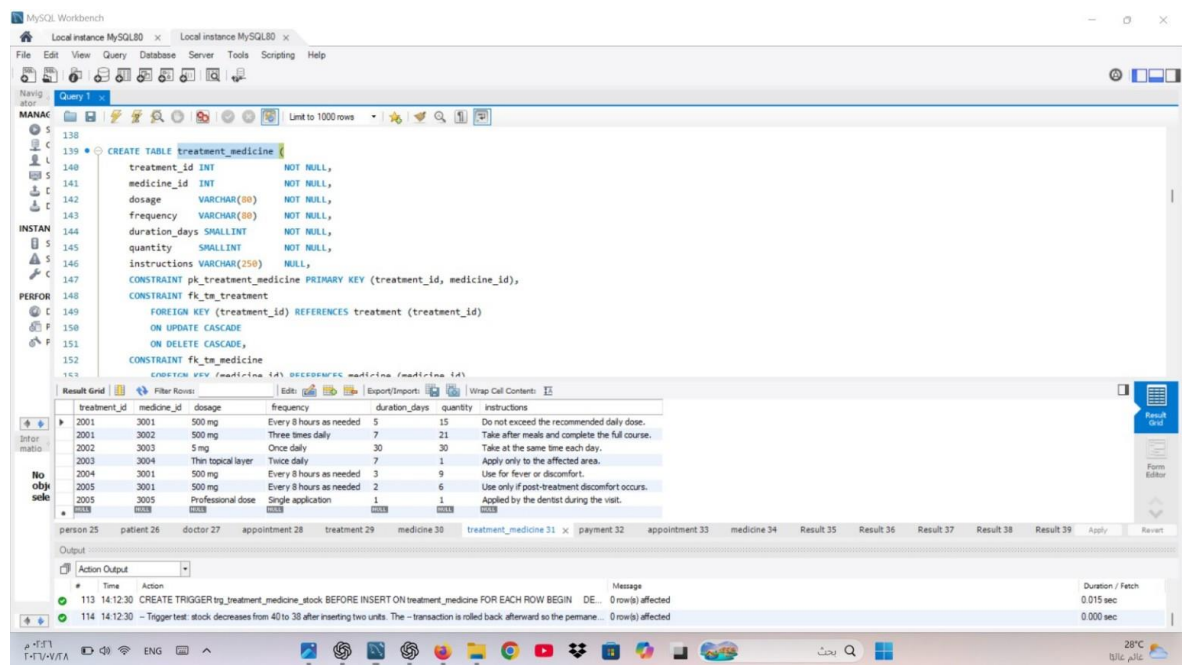
Screenshot 2.4. APPOINTMENT table definition and five populated appointment records.



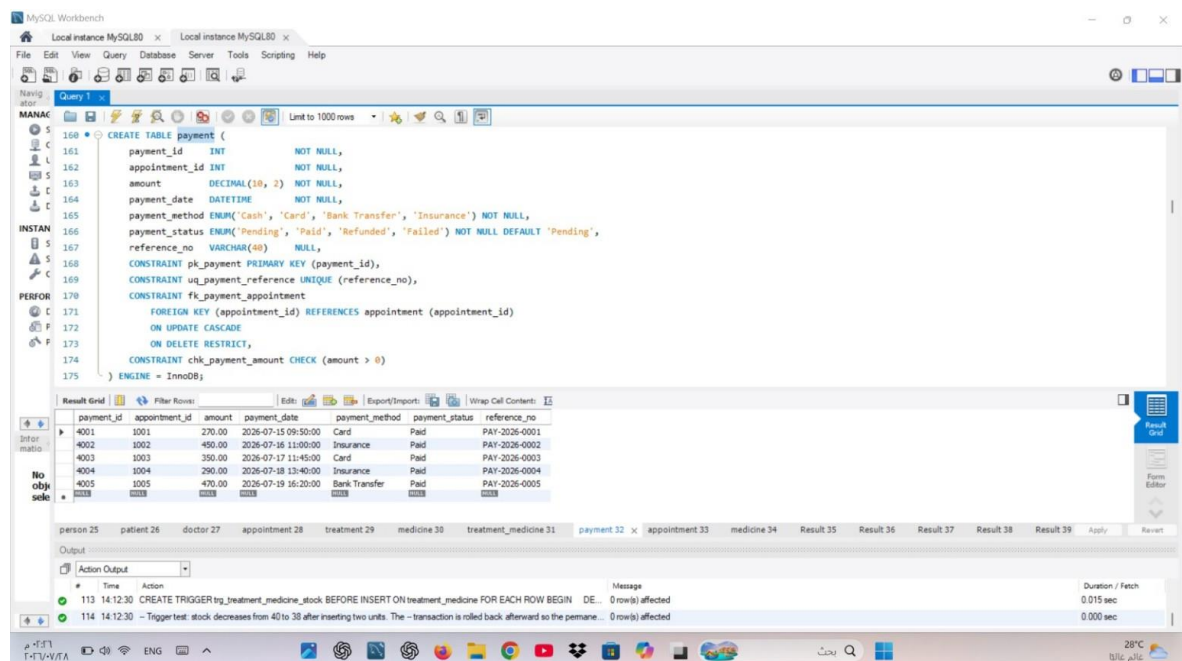
Screenshot 2.5. TREATMENT table definition and five populated treatment records.



Screenshot 2.6. MEDICINE table definition and five populated medicine records.



Screenshot 2.7. TREATMENT_MEDICINE definition and seven populated prescription-detail records.



Screenshot 2.8. PAYMENT table definition and five completed payment records.

3. Task 3 - SQL Operations

Every assessed operation was executed in MySQL Workbench. Purpose statements, SQL excerpts, and matching outputs are provided below. UPDATE, DELETE, and trigger demonstrations use transactions and ROLLBACK so the original academic dataset remains unchanged.

3.1 Verified Result Summary

Operation	Verified MySQL Result
SELECT	Five completed appointments; two medicines with stock at or below 100.
JOIN	Five appointment summaries and seven patient-prescription rows.
Nested query	Faisal (SAR 470.00) and Mohammed (SAR 450.00).
GROUP BY	Five doctor workload and financial summary rows.
UPDATE	Hydrocortisone price 18.00 -> 18.90; restored by rollback.
DELETE	Matching row count 1 -> 0 -> 1 after rollback.
VIEW	vw_appointment_summary exists and returns five complete rows.
TRIGGER	Trigger exists; stock 40 -> 38 -> 40 after rollback.

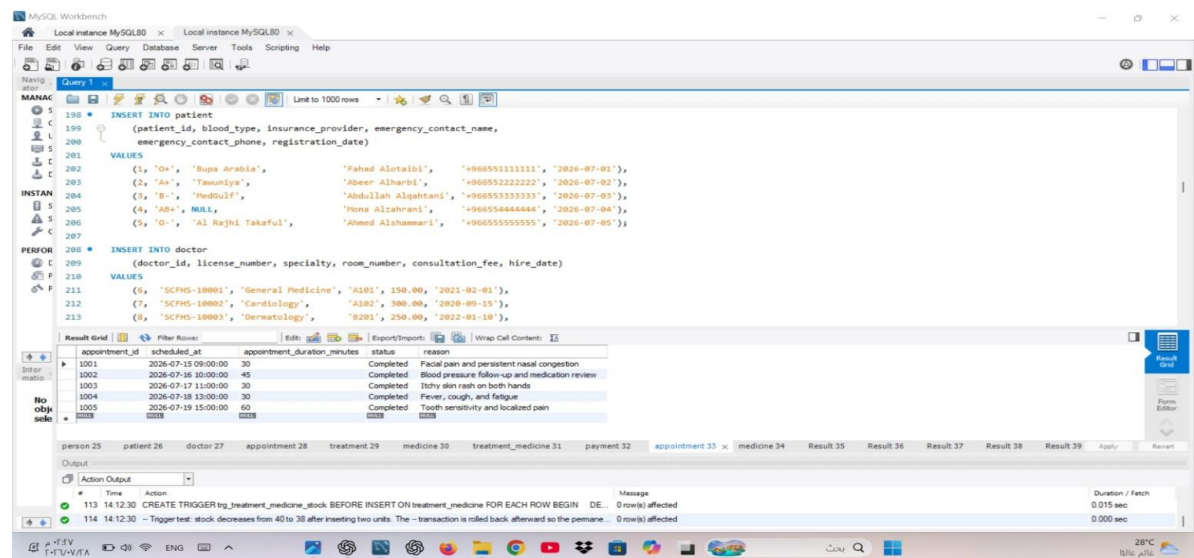
3.2 SELECT Statements

Purpose: List completed appointments chronologically and identify medicines requiring closer stock monitoring.

SQL code

```
SELECT
    appointment_id,
    scheduled_at,
    appointment_duration_minutes,
    status,
    reason
FROM appointment
WHERE status = 'Completed'
ORDER BY scheduled_at;

SELECT
    medicine_id,
    medicine_name,
    strength,
    stock_quantity,
    unit_price
FROM medicine
WHERE stock_quantity <= 100
ORDER BY stock_quantity ASC, medicine_name ASC;
```



Screenshot 3.1A. Completed appointments returned by the SELECT statement.

medicine_id	medicine_name	strength	stock_quantity	unit_price
3005	Sodium Fluoride Gel	1.23%	40	25.00
3004	Hydrocortisone	1%	60	18.00

Screenshot 3.1B. Medicines with stock quantities of 100 units or fewer.

3.3 JOIN Query - Appointment Details

Purpose: Combine each appointment with readable patient, doctor, and specialty details.

SQL code

```
SELECT
    a.appointment_id,
    a.scheduled_at,
    patient_person.full_name AS patient_name,
    doctor_person.full_name AS doctor_name,
    d.specialty,
    a.status
FROM appointment AS a
JOIN patient AS pat ON pat.patient_id = a.patient_id
JOIN person AS patient_person ON patient_person.person_id = pat.patient_id
JOIN doctor AS d ON d.doctor_id = a.doctor_id
JOIN person AS doctor_person ON doctor_person.person_id = d.doctor_id
ORDER BY a.scheduled_at;
```

appointment_id	scheduled_at	patient_name	doctor_name	specialty	status
1001	2026-07-15 09:00:00	Noura Fahad Alotaibi	Huda Saleh Almutairi	General Medicine	Completed
1002	2026-07-16 10:00:00	Mohammed Saad Alharbi	Fahad Nasser Alghamdi	Cardiology	Completed
1003	2026-07-17 11:00:00	Reem Abdullah Alqahitani	Maha Khalid Aldosari	Dermatology	Completed
1004	2026-07-18 13:00:00	Layan Ahmed Alshammari	Omar Ali Alqahitani	Pediatrics	Completed
1005	2026-07-19 15:00:00	Faisal Omar Alzahrani	Sara Ibrahim Alsubaie	Dentistry	Completed

Screenshot 3.2A. Appointment details JOIN result.

3.4 JOIN Query - Patient Prescriptions

Purpose: Connect patients to diagnoses and prescribed medicines, including dosage, frequency, and duration.

SQL code

```
SELECT
    patient_person.full_name AS patient_name,
    t.diagnosis,
    m.medicine_name,
    m.strength,
    tm.dosage,
    tm.frequency,
    tm.duration_days
FROM patient AS pat
JOIN person AS patient_person ON patient_person.person_id = pat.patient_id
JOIN appointment AS a ON a.patient_id = pat.patient_id
JOIN treatment AS t ON t.appointment_id = a.appointment_id
JOIN treatment_medicine AS tm ON tm.treatment_id = t.treatment_id
JOIN medicine AS m ON m.medicine_id = tm.medicine_id
ORDER BY patient_person.full_name, m.medicine_name;
```

The screenshot displays the MySQL Workbench interface. The top pane shows the SQL query for joining patient, appointment, treatment, and medicine tables. The bottom pane shows the results of the query, which is a table with 7 columns: patient_name, diagnosis, medicine_name, strength, dosage, frequency, and duration_days. The results table contains 7 rows of data for various patients and their treatments.

patient_name	diagnosis	medicine_name	strength	dosage	frequency	duration_days
Faisal Omar Alazhri	Early dental caries	Paracetamol	500 mg	500 mg	Every 8 hours as needed	2
Faisal Omar Alazhri	Early dental caries	Sodium Fluoride Gel	1.23%	Professional dose	Single application	1
Layla Ahmed Alahmari	Seasonal influenza	Paracetamol	500 mg	500 mg	Every 8 hours as needed	3
Mohammed Saad Alharbi	Essential hypertension - controlled	Amlodipine	5 mg	5 mg	Once daily	30
Houira Fahad Alotabi	Acute bacterial sinusitis	Amoxicillin	500 mg	500 mg	Three times daily	7
Houira Fahad Alotabi	Acute bacterial sinusitis	Paracetamol	500 mg	500 mg	Every 8 hours as needed	5
Reem Abdullah Alghamdi	Mild contact dermatitis	Hydrocortisone	1%	Thin topical layer	Twice daily	7

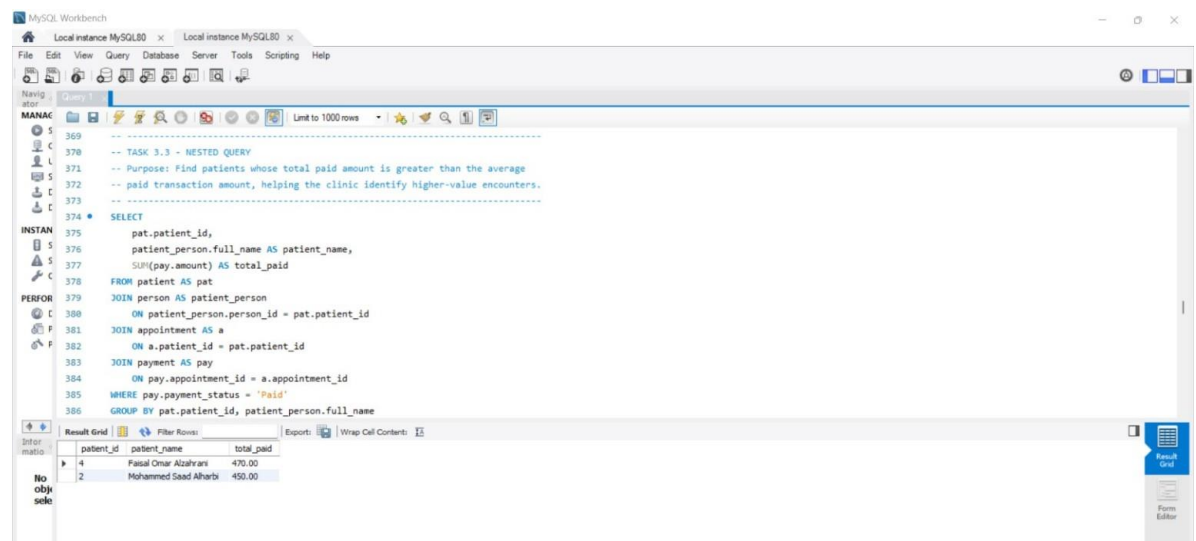
Screenshot 3.2B. Patient prescription JOIN query and result.

3.5 Nested Query

Purpose: Identify patients whose total paid amount is greater than the average paid transaction.

SQL code

```
SELECT
    pat.patient_id,
    patient_person.full_name AS patient_name,
    SUM(pay.amount) AS total_paid
FROM patient AS pat
JOIN person AS patient_person ON patient_person.person_id = pat.patient_id
JOIN appointment AS a ON a.patient_id = pat.patient_id
JOIN payment AS pay ON pay.appointment_id = a.appointment_id
WHERE pay.payment_status = 'Paid'
GROUP BY pat.patient_id, patient_person.full_name
HAVING SUM(pay.amount) > (
    SELECT AVG(paid_payment.amount)
    FROM payment AS paid_payment
    WHERE paid_payment.payment_status = 'Paid'
)
ORDER BY total_paid DESC;
```



Screenshot 3.3. Nested query and the two qualifying patients.

3.6 Aggregate Functions with GROUP BY

Purpose: Summarize doctor workload, average appointment duration, treatment cost, and collected payments.

SQL code

```
SELECT
    d.doctor_id,
    doctor_person.full_name AS doctor_name,
    d.specialty,
    COUNT(DISTINCT a.appointment_id) AS appointment_count,
    ROUND(AVG(a.appointment_duration_minutes), 2) AS average_duration_minutes,
    SUM(t.treatment_cost) AS total_treatment_cost,
    SUM(CASE WHEN pay.payment_status = 'Paid' THEN pay.amount ELSE 0 END)
    AS total_collected
FROM doctor AS d
JOIN person AS doctor_person ON doctor_person.person_id = d.doctor_id
LEFT JOIN appointment AS a ON a.doctor_id = d.doctor_id
LEFT JOIN treatment AS t ON t.appointment_id = a.appointment_id
LEFT JOIN payment AS pay ON pay.appointment_id = a.appointment_id
GROUP BY d.doctor_id, doctor_person.full_name, d.specialty
ORDER BY appointment_count DESC, doctor_name;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the query from the previous block. The 'Result Grid' tab is active, displaying the results of the query. The results are summarized in the table below:

doctor_id	doctor_name	specialty	appointment_count	average_duration_minutes	total_treatment_cost	total_collected
6	Huda Saleh Almutairi	General Medicine	1	30.00	120.00	270.00
7	Fahad Nasser Alghamdi	Cardiology	1	45.00	150.00	450.00
8	Maha Khalid Aldosari	Dermatology	1	30.00	100.00	350.00
9	Omar Ali Alqahitani	Pediatrics	1	30.00	90.00	290.00
10	Sara Ibrahim Alsubaie	Dentistry	1	60.00	250.00	470.00

The bottom of the screenshot shows the 'History Output' tab with a list of SQL commands executed, including table creation and naming commands.

Screenshot 3.4. Aggregate query with GROUP BY and five doctor summaries.

3.7 UPDATE Statement

Purpose: Demonstrate a five-percent Hydrocortisone price adjustment without permanently changing the academic dataset.

SQL code

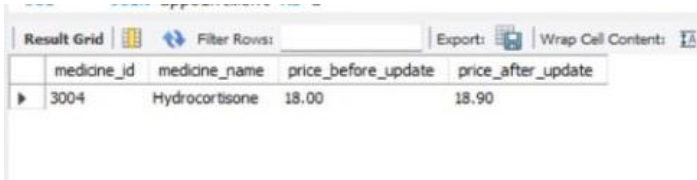
```
START TRANSACTION;

SET @original_hydrocortisone_price = (
    SELECT unit_price FROM medicine WHERE medicine_id = 3004
);

UPDATE medicine
SET unit_price = ROUND(unit_price * 1.05, 2)
WHERE medicine_id = 3004;

SELECT
    medicine_id,
    medicine_name,
    @original_hydrocortisone_price AS price_before_update,
    unit_price AS price_after_update
FROM medicine
WHERE medicine_id = 3004;

ROLLBACK;
```



The screenshot shows a SQL result grid with the following data:

medicine_id	medicine_name	price_before_update	price_after_update
3004	Hydrocortisone	18.00	18.90

Screenshot 3.5. Hydrocortisone price before and after the UPDATE statement.

3.8 DELETE Statement and Rollback Verification

Purpose: Demonstrate removal of one optional treatment-medicine detail row and restore it after verification.

Evidence verification SQL

```
SET @before_count = (
```


3.9 VIEW Creation and Result

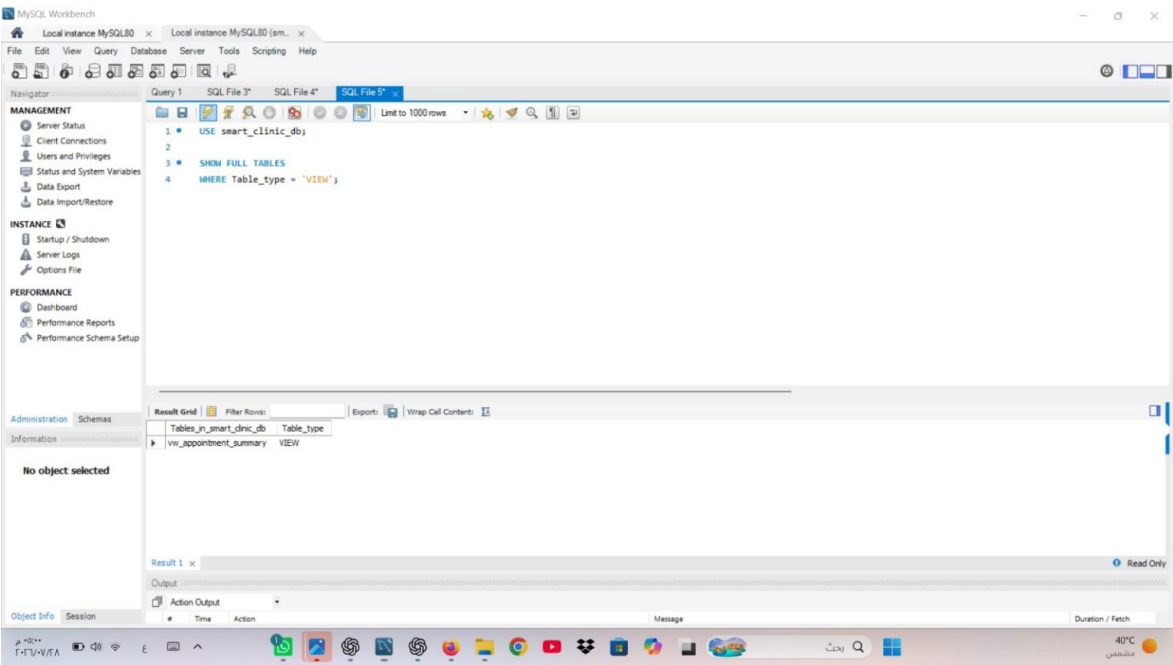
Purpose: Create a reusable appointment summary combining patient, doctor, treatment, and paid-payment information.

SQL code

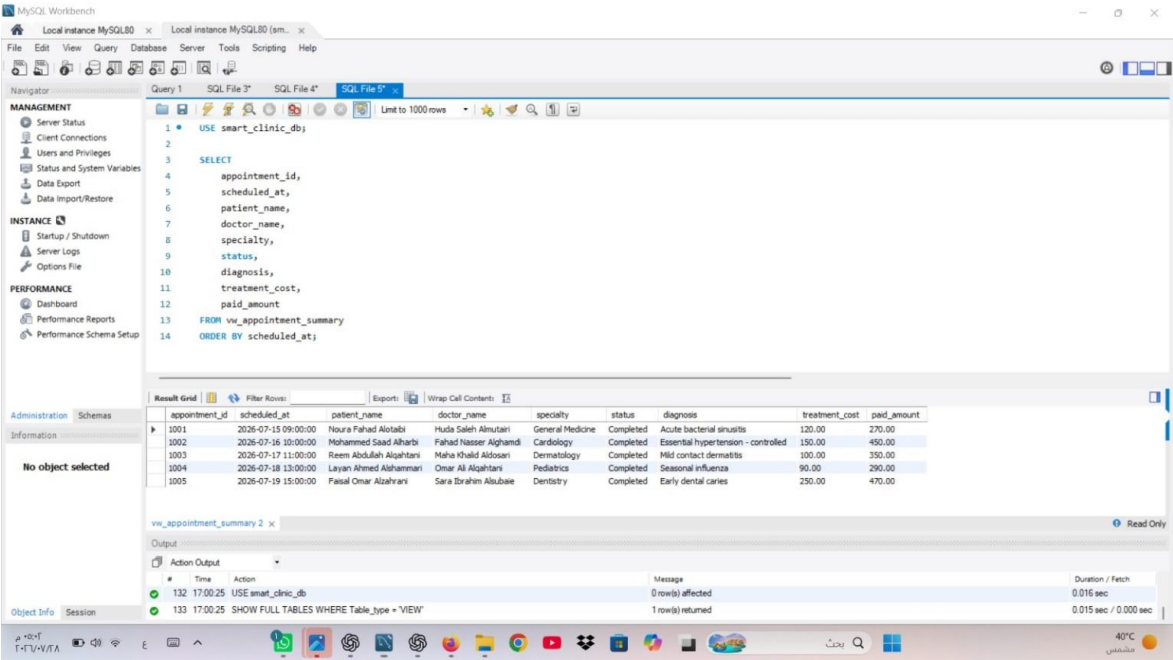
```
DROP VIEW IF EXISTS vw_appointment_summary;

CREATE VIEW vw_appointment_summary AS
SELECT
    a.appointment_id,
    a.scheduled_at,
    patient_person.full_name AS patient_name,
    doctor_person.full_name AS doctor_name,
    d.specialty,
    a.status,
    t.diagnosis,
    t.treatment_cost,
    COALESCE(SUM(CASE WHEN pay.payment_status = 'Paid'
        THEN pay.amount ELSE 0 END), 0.00) AS paid_amount
FROM appointment AS a
JOIN patient AS pat ON pat.patient_id = a.patient_id
JOIN person AS patient_person ON patient_person.person_id = pat.patient_id
JOIN doctor AS d ON d.doctor_id = a.doctor_id
JOIN person AS doctor_person ON doctor_person.person_id = d.doctor_id
LEFT JOIN treatment AS t ON t.appointment_id = a.appointment_id
LEFT JOIN payment AS pay ON pay.appointment_id = a.appointment_id
GROUP BY
    a.appointment_id, a.scheduled_at, patient_person.full_name,
    doctor_person.full_name, d.specialty, a.status,
    t.diagnosis, t.treatment_cost;

SELECT *
FROM vw_appointment_summary
ORDER BY scheduled_at;
```



Screenshot 3.7A. MySQL confirms vw_appointment_summary as a VIEW.



Screenshot 3.7B. Full five-row result returned from vw_appointment_summary.

4.10 TRIGGER Creation and Stock-Control Test

Purpose: Protect medicine inventory by validating stock and automatically reducing it before a treatment-medicine insert.

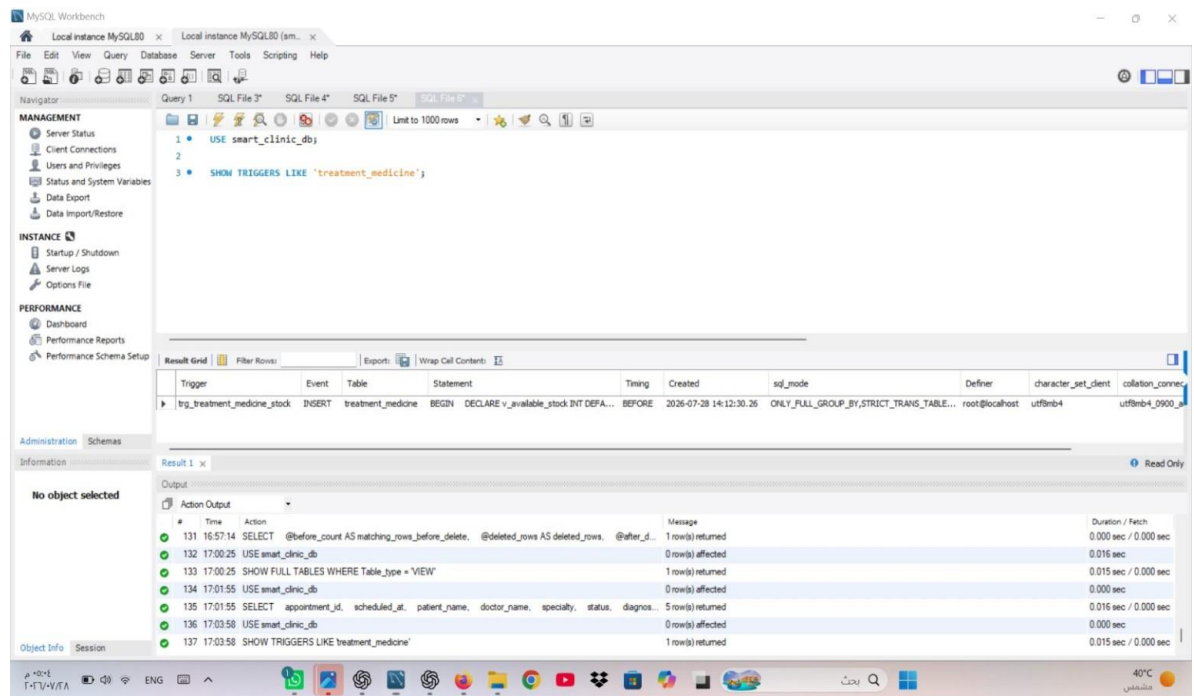
Trigger definition

```
DROP TRIGGER IF EXISTS trg_treatment_medicine_stock;

DELIMITER $$
CREATE TRIGGER trg_treatment_medicine_stock
BEFORE INSERT ON treatment_medicine
FOR EACH ROW
BEGIN
    DECLARE v_available_stock INT DEFAULT -1;

    SELECT COALESCE(MAX(stock_quantity), -1)
    INTO v_available_stock
    FROM medicine
    WHERE medicine_id = NEW.medicine_id;

    IF v_available_stock = -1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'The selected medicine does not exist.';
    ELSEIF NEW.quantity > v_available_stock THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient medicine stock for this treatment.';
    ELSE
        UPDATE medicine
        SET stock_quantity = stock_quantity - NEW.quantity
        WHERE medicine_id = NEW.medicine_id;
    END IF;
END$$
DELIMITER ;
```



Screenshot 3.8A. SHOW TRIGGERS result confirming trg_treatment_medicine_stock.

Trigger test and rollback verification

```

SET @stock_before_trigger = (
    SELECT stock_quantity FROM medicine WHERE medicine_id = 3005
);

```

```
START TRANSACTION;
```

```

INSERT INTO treatment_medicine
    (treatment_id, medicine_id, dosage, frequency,
     duration_days, quantity, instructions)
VALUES
    (2004, 3005, 'Professional dose', 'Single application',
     1, 2, 'Temporary row used to verify the stock-control trigger.');
```

```

SET @stock_after_trigger = (
    SELECT stock_quantity FROM medicine WHERE medicine_id = 3005
);

```

```
ROLLBACK;
```

```

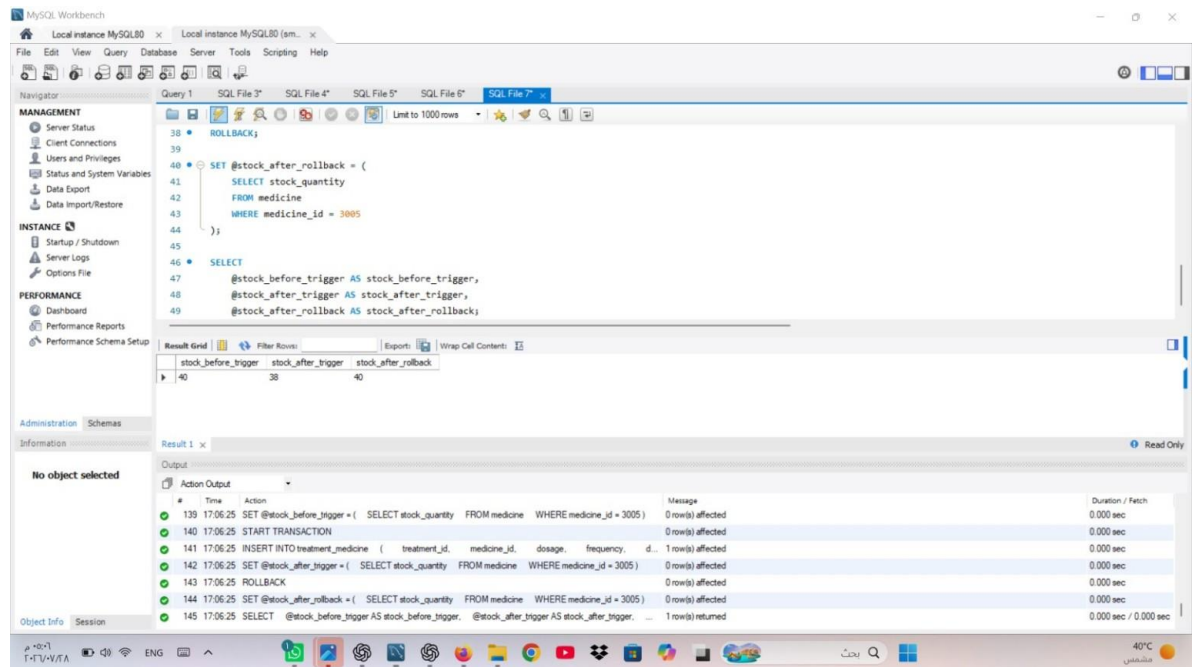
SET @stock_after_rollback = (
    SELECT stock_quantity FROM medicine WHERE medicine_id = 3005
);

```

```
);
```

```
SELECT
```

```
  @stock_before_trigger AS stock_before_trigger,  
  @stock_after_trigger AS stock_after_trigger,  
  @stock_after_rollback AS stock_after_rollback;
```



Screenshot 3.8B. Trigger test result showing stock 40 -> 38 -> 40.

4. Task 4 – Project Reflection

Developing the Smart Clinic Database System was a practical team experience that required us to connect database theory with a realistic clinic workflow. Our most significant shared challenge was preserving consistency while integrating design, implementation, SQL operations, and screenshot evidence into one repeatable MySQL script.

We identified the problem when foreign-key dependencies, result tabs, and temporary UPDATE, DELETE, and trigger tests had to remain synchronized. We resolved it by creating and inserting data in parent-to-child order, testing the complete script in MySQL Workbench, using transactions for reversible changes, and reviewing every screenshot against its corresponding SQL operation.

Dana Mohammed Alajmi: As Group Leader and Repository Manager, Dana coordinated the work, reviewed the integrated files, organized the evidence, and prepared the submission structure. She chose one evolving SQL file instead of separate scripts for each task. Separate files would have been easier for individual editing, but they could have produced conflicting table definitions or sample data. The single-file approach required more careful merging, yet it met the submission requirement and gave the team one authoritative implementation.

AMANI BARJAS: Amani completed the database design and selected PERSON as a supertype specialized into PATIENT and DOCTOR. The alternative was to store repeated identity and contact fields separately in both tables. The selected approach introduced additional joins when retrieving full records, but it reduced redundancy, improved consistency, and demonstrated the required EER specialization. She also reviewed keys, cardinalities, assumptions, and the relational mapping before implementation.

Dareen Alanazi: Dareen implemented the database tables, constraints, and fictional Saudi-context records. Her main obstacle was maintaining valid references across eight tables. She resolved it by creating parent tables first, inserting consistent identifiers in dependency order, and verifying every populated table. UNIQUE, CHECK, and foreign-key constraints were used to detect invalid or duplicated values early.

Manar Mutaib Al-Harbi: Manar prepared and tested the SELECT, JOIN, nested, GROUP BY, UPDATE, DELETE, VIEW, and TRIGGER operations. Because UPDATE, DELETE, and trigger tests could alter the original dataset, she used START TRANSACTION and ROLLBACK. The final evidence confirmed the DELETE sequence 1 to 0 to 1 and the trigger stock sequence 40 to 38 to 40.

If we repeated the project with more time, we would create a formal test checklist before implementation, capture screenshots during the first verified execution, and add audit logging or stored procedures for common clinic operations. AI-based tools were used only for limited wording review, document organization, and SQL syntax suggestions. The team made the design decisions, executed the database, verified the results, and captured the MySQL evidence.

5. Project Artifacts

The final URLs should be inserted in the highlighted cells before the report is exported to PDF and submitted through Blackboard.

Artifact	Final Entry
Live report document (with edit history)	https://drive.google.com/drive/folders/1m4mQ34nHjkG3mkRDAtZNPv_B7Xkpg7DG
GitHub/GitLab repository (with commit history)	https://github.com/dana2002-cell/IT244-Project-
SQL script	database/smart_clinic_database.sql
ER/EER diagram	diagrams/er_diagram.png
MySQL evidence	evidence/screenshots/ 20.jpg

6. Appendix A - Mid-Project Progress Report

Smart_Clinic_Mid_Project_Progres_Report

Students Details

Student Name	Student ID	Primary Contribution
Dana Mohammed Alajmi	240040990	Group Leader and Repository Manager
Amani Barjas	240047827	Task 1: Database Design
Dareen Alanazi	240022387	Task 2: Database Implementation
Manar Mutaib Al-Harbi	240061219	Task 3: SQL Operations

Progress Summary

The project has completed the Task 3 preparation stage. The single MySQL file now contains the required SELECT, JOIN, nested, GROUP BY, UPDATE, DELETE, VIEW, and TRIGGER operations with purpose statements. The file passed MySQL parsing and internal data checks, and equivalent operations returned coherent results in an isolated Supabase PostgreSQL validation schema. Actual MySQL Workbench execution and course-compliant screenshot capture remain required before final approval

Workstream	Status	Evidence/Output
Task 1 - Design	Prepared for team review	ER/EER diagram, relational schema, keys, cardinalities, assumptions
Task 2 - Implementation	Prepared for MySQL execution	Database, eight tables, constraints, and sample records
Task 2 Evidence	Pending actual capture	MySQL Workbench code, table outputs, and execution status
Task 3 - SQL Operations	Implemented and support-validated	MySQL code plus eight genuine, clearly labeled Supabase result panels
Task 3 MySQL Evidence	Pending actual capture	MySQL Workbench code and output screenshots for every assessed operation
Task 4 - Reflection	Deferred by instruction	Will be written only after the full project is complete

Completed Work

- Defined eight relational tables covering people, patients, doctors, appointments, treatments, medicines, prescriptions, and payments.
- Modeled PERSON as a supertype specialized into the PATIENT and DOCTOR subtypes.
- Specified primary keys, foreign keys, uniqueness constraints, checks, and controlled ENUM values.

- Produced a complete ER/EER diagram with entity attributes and relationship cardinalities.
- Prepared a repeatable MySQL 8.0 script with a transaction-based insertion sequence.
- Inserted at least five logically connected records into every main table.
- Added SELECT, JOIN, nested, GROUP BY, UPDATE, DELETE, VIEW, and TRIGGER operations to the same SQL file.
- Executed equivalent Task 3 operations in an isolated Supabase schema and restored all temporary data changes.
- Created eight genuine evidence panels that identify their platform and do not impersonate MySQL Workbench.

Key Design Decisions

Decision	Reason
Shared person supertype	Avoids repeated identity and contact attributes in separate patient and doctor tables.
Disjoint, total specialization	Makes each clinic person one clear role while demonstrating the required EER feature.
Associative prescription table	Stores dosage and duration while resolving the treatment-medicine many-to-many relationship.
Multiple payments per appointment	Supports insurance, installments, and future mixed payment scenarios.
Restrictive transactional deletion	Protects clinical and financial history from accidental parent-record deletion.
One evolving SQL file	Meets the final requirement for a single implementation file and prevents inconsistent copies.

Challenges and Responses

Design-Stage Challenge	Response
Representing specialization relationally	Used PERSON as the supertype and made each subtype key a foreign key to PERSON.
Maintaining referential insertion order	Created and populated tables from parent entities to dependent transactional entities inside one transaction.
Preventing schedule conflicts	Added unique doctor-time and patient-time constraints to APPOINTMENT.
Protecting evidence integrity	Kept MySQL placeholders, added visibly labeled supporting Supabase results, and avoided presenting them as Workbench evidence.
Trigger portability	Qualified the PostgreSQL validation function's schema, set an immutable search path, and retained the correct MySQL trigger syntax in the assessed file.

Team Contributions

Member	Current Contribution	Status
Dana Mohammed Alajmi	Project coordination, file organization, repository management, integration review	In progress
Amani Barjas	Task 1 entity analysis, keys, relationships, cardinalities, EER feature, assumptions, relational schema	Prepared for review

Member	Current Contribution	Status
Dareen Alanazi	Task 2 database creation, table definitions, constraints, data insertion, and testing preparation	Prepared for MySQL execution
Manar Mutaib Al-Harbi	Task 3 SELECT, JOIN, nested, aggregate, UPDATE, DELETE, VIEW, TRIGGER, purposes, and evidence specification	Prepared for team review
All members	Task 4 Reflection, Cross-review, testing, evidence, progress report, final report.	Ongoing

Next-Stage Plan

- We will review the ER/EER model, assumptions, relational schema, sample data, and Task 3 purposes as a team.
- We will run the complete Tasks 1–3 script in MySQL Workbench and correct any environment-specific issues.
- We will capture and organize all required Task 2 and Task 3 MySQL Workbench screenshots.
- We will compare the MySQL results with the supporting validation evidence and approve the final technical results.
- We will review the actual MySQL query results before writing the 300–500-word Project Reflection.
- We will assemble and verify the final DOCX, PDF, correct repository link, and compressed submission package.

7. Appendix B - Complete SQL Implementation

The following is the complete MySQL implementation submitted with the project. It includes database creation, eight tables, sample data, verification queries, required Task 3 operations, the view, and the trigger.

[database/smart_clinic_database.sql](#)

```

/*
=====
Smart Clinic Database System IT244 - Database Design and
Implementation Project Stage 2 Scope: Task 1 (Design), Task 2
(Implementation), and Task 3 (SQL Operations) Task 4 Project
Reflection remains intentionally excluded until final review.

Team Members
1. Dana Mohammed Alajmi -240040990 - Group Leader and Repository
Manager
2. AMANI BARJAS - 240047827 - Task 1: Database Design
3. Dareen Alanazi - 240022387 - Task 2: Database Implementation
4. Manar Mutaib Al-Harbi - 240061219 - Task 3: SQL Operations

Target DBMS: MySQL 8.0+ All personal and clinical data below are
fictional and created for academic use.
===== */

-- Recreate the database so the script can be executed repeatedly
during testing.
DROP DATABASE IF EXISTS smart_clinic_db;
CREATE DATABASE smart_clinic_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_0900_ai_ci;

USE smart_clinic_db;
SET NAMES utf8mb4;

-----
-- TASK 1 RELATIONAL MAPPING AND TASK 2 TABLE IMPLEMENTATION
-- EER feature: PERSON is a supertype specialized into PATIENT and
DOCTOR.
-- The specialization is total and disjoint as an application-level
rule.

-----

```

```

CREATE TABLE person (
  person_id          INT          NOT NULL,
  national_id        VARCHAR(10)  NOT NULL,
  full_name          VARCHAR(100) NOT NULL,
  date_of_birth      DATE          NOT NULL,
  gender             ENUM('Female', 'Male') NOT NULL,
  phone              VARCHAR(13)   NOT NULL,
  email              VARCHAR(120)  NULL,
  address            VARCHAR(200)  NOT NULL,
  CONSTRAINT pk_person PRIMARY KEY (person_id),
  CONSTRAINT uq_person_national_id UNIQUE (national_id),
  CONSTRAINT uq_person_phone UNIQUE (phone),
  CONSTRAINT uq_person_email UNIQUE (email),
  CONSTRAINT chk_person_national_id CHECK (national_id REGEXP
'^[12][0-9]{9}$'),
  CONSTRAINT chk_person_phone CHECK (phone REGEXP '^\\+9665[0-9]{8}$')
) ENGINE = InnoDB;

CREATE TABLE patient (
  patient_id          INT          NOT NULL,
  blood_type          ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-',
'O+', 'O-') NOT NULL,
  insurance_provider  VARCHAR(100) NULL,
  emergency_contact_name VARCHAR(100) NOT NULL,
  emergency_contact_phone VARCHAR(13) NOT NULL,
  registration_date   DATE          NOT NULL,
  CONSTRAINT pk_patient PRIMARY KEY (patient_id),
  CONSTRAINT fk_patient_person
    FOREIGN KEY (patient_id) REFERENCES person (person_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT chk_patient_emergency_phone
    CHECK (emergency_contact_phone REGEXP '^\\+9665[0-9]{8}$')
) ENGINE = InnoDB;

CREATE TABLE doctor (
  doctor_id          INT          NOT NULL,
  license_number      VARCHAR(25)  NOT NULL,
  specialty           VARCHAR(80)  NOT NULL,
  room_number         VARCHAR(10)  NOT NULL,
  consultation_fee     DECIMAL(10, 2) NOT NULL,
  hire_date           DATE          NOT NULL,
  CONSTRAINT pk_doctor PRIMARY KEY (doctor_id),
  CONSTRAINT uq_doctor_license UNIQUE (license_number),
  CONSTRAINT uq_doctor_room UNIQUE (room_number),
  CONSTRAINT fk_doctor_person
    FOREIGN KEY (doctor_id) REFERENCES person (person_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT chk_doctor_fee CHECK (consultation_fee >= 0)

```

```

) ENGINE = InnoDB;

CREATE TABLE appointment (
    appointment_id          INT          NOT NULL,
    patient_id              INT          NOT NULL,
    doctor_id               INT          NOT NULL,
    scheduled_at            DATETIME     NOT NULL,
    appointment_duration_minutes SMALLINT NOT NULL DEFAULT 30,
    status                  ENUM('Scheduled', 'Completed',
    'Cancelled', 'No Show') NOT NULL DEFAULT 'Scheduled',
    reason                  VARCHAR(250) NOT NULL,
    created_at              TIMESTAMP    NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    CONSTRAINT pk_appointment PRIMARY KEY (appointment_id),
    CONSTRAINT uq_appointment_doctor_slot UNIQUE (doctor_id,
scheduled_at),
    CONSTRAINT uq_appointment_patient_slot UNIQUE (patient_id,
scheduled_at),
    CONSTRAINT fk_appointment_patient
        FOREIGN KEY (patient_id) REFERENCES patient (patient_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_appointment_doctor
        FOREIGN KEY (doctor_id) REFERENCES doctor (doctor_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT chk_appointment_duration
        CHECK (appointment_duration_minutes BETWEEN 15 AND 180)
) ENGINE = InnoDB;

CREATE TABLE treatment (
    treatment_id          INT          NOT NULL,
    appointment_id        INT          NOT NULL,
    treatment_date         DATE         NOT NULL,
    diagnosis              VARCHAR(200) NOT NULL,
    treatment_notes        VARCHAR(500) NOT NULL,
    treatment_cost         DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    CONSTRAINT pk_treatment PRIMARY KEY (treatment_id),
    CONSTRAINT uq_treatment_appointment UNIQUE (appointment_id),
    CONSTRAINT fk_treatment_appointment
        FOREIGN KEY (appointment_id) REFERENCES appointment
(appointment_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT chk_treatment_cost CHECK (treatment_cost >= 0)
) ENGINE = InnoDB;

CREATE TABLE medicine (
    medicine_id          INT          NOT NULL,
    medicine_name         VARCHAR(100) NOT NULL,
    strength              VARCHAR(30)  NOT NULL,

```

```

dosage_form          VARCHAR(40)          NOT NULL,
unit_price           DECIMAL(10, 2)       NOT NULL,
stock_quantity       INT                  NOT NULL,
prescription_required BOOLEAN            NOT NULL DEFAULT TRUE,
CONSTRAINT pk_medicine PRIMARY KEY (medicine_id),
CONSTRAINT uq_medicine_name_strength UNIQUE (medicine_name,
strength),
CONSTRAINT chk_medicine_price CHECK (unit_price >= 0),
CONSTRAINT chk_medicine_stock CHECK (stock_quantity >= 0)
) ENGINE = InnoDB;

CREATE TABLE treatment_medicine (
  treatment_id INT          NOT NULL,
  medicine_id  INT          NOT NULL,
  dosage       VARCHAR(80)  NOT NULL,
  frequency    VARCHAR(80)  NOT NULL,
  duration_days SMALLINT    NOT NULL,
  quantity     SMALLINT     NOT NULL,
  instructions VARCHAR(250)  NULL,
  CONSTRAINT pk_treatment_medicine PRIMARY KEY (treatment_id,
medicine_id),
  CONSTRAINT fk_tm_treatment
    FOREIGN KEY (treatment_id) REFERENCES treatment (treatment_id)
    ON UPDATE CASCADE
    ON DELETE CASCADE,
  CONSTRAINT fk_tm_medicine
    FOREIGN KEY (medicine_id) REFERENCES medicine (medicine_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT chk_tm_duration CHECK (duration_days > 0),
  CONSTRAINT chk_tm_quantity CHECK (quantity > 0)
) ENGINE = InnoDB;

CREATE TABLE payment (
  payment_id      INT          NOT NULL,
  appointment_id  INT          NOT NULL,
  amount          DECIMAL(10, 2) NOT NULL,
  payment_date    DATETIME     NOT NULL,
  payment_method  ENUM('Cash', 'Card', 'Bank Transfer', 'Insurance')
NOT NULL,
  payment_status  ENUM('Pending', 'Paid', 'Refunded', 'Failed') NOT
NULL DEFAULT 'Pending',
  reference_no    VARCHAR(40)    NULL,
  CONSTRAINT pk_payment PRIMARY KEY (payment_id),
  CONSTRAINT uq_payment_reference UNIQUE (reference_no),
  CONSTRAINT fk_payment_appointment
    FOREIGN KEY (appointment_id) REFERENCES appointment
(appointment_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
  CONSTRAINT chk_payment_amount CHECK (amount > 0)

```

```

) ENGINE = InnoDB;

-----
-- TASK 2 SAMPLE DATA
-- Ten PERSON rows support five PATIENT subtypes and five DOCTOR
subtypes.
-----

START TRANSACTION;

INSERT INTO person
    (person_id, national_id, full_name, date_of_birth, gender, phone,
    email, address)
VALUES
    (1, '1098765432', 'Noura Fahad Alotaibi', '1993-04-18',
    'Female', '+966501234567', 'noura.alotaibi@example.sa', 'Al Malqa
    District, Riyadh'),
    (2, '1087654321', 'Mohammed Saad Alharbi', '1986-09-03',
    'Male', '+966502345678', 'mohammed.alharbi@example.sa', 'Al
    Yasmin District, Riyadh'),
    (3, '1076543219', 'Reem Abdullah Alqahtani', '1998-12-11',
    'Female', '+966503456789', 'reem.alqahtani@example.sa', 'Al
    Narjis District, Riyadh'),
    (4, '1065432198', 'Faisal Omar Alzahrani', '1979-02-26',
    'Male', '+966504567890', 'faisal.alzahrani@example.sa', 'Al
    Rawdah District, Riyadh'),
    (5, '1123456789', 'Layan Ahmed Alshammari', '2014-06-07',
    'Female', '+966505678901', 'layan.alshammari@example.sa', 'Al
    Qurtubah District, Riyadh'),
    (6, '1054321987', 'Huda Saleh Almutairi', '1982-01-14',
    'Female', '+966506789012', 'huda.almutairi@smartclinic.sa', 'Al
    Wurud District, Riyadh'),
    (7, '1043219876', 'Fahad Nasser Alghamdi', '1978-08-22',
    'Male', '+966507890123', 'fahad.alghamdi@smartclinic.sa', 'Al
    Olaya District, Riyadh'),
    (8, '1032198765', 'Maha Khalid Aldosari', '1987-05-30',
    'Female', '+966508901234', 'maha.aldosari@smartclinic.sa', 'Al
    Nakheel District, Riyadh'),
    (9, '1021987654', 'Omar Ali Alqahtani', '1984-10-09',
    'Male', '+966509012345', 'omar.alqahtani@smartclinic.sa', 'Al
    Sahafah District, Riyadh'),
    (10, '1019876543', 'Sara Ibrahim Alsubaie', '1981-03-17',
    'Female', '+966550123456', 'sara.alsubaie@smartclinic.sa', 'Al
    Rabwah District, Riyadh');

INSERT INTO patient
    (patient_id, blood_type, insurance_provider,
    emergency_contact_name,
    emergency_contact_phone, registration_date)

```


VALUES

```
(1, 'O+', 'Bupa Arabia', 'Fahad Alotaibi',
'+966551111111', '2026-07-01'),
(2, 'A+', 'Tawuniya', 'Abeer Alharbi',
'+966552222222', '2026-07-02'),
(3, 'B-', 'MedGulf', 'Abdullah Alqahtani',
'+966553333333', '2026-07-03'),
(4, 'AB+', NULL, 'Mona Alzahrani',
'+966554444444', '2026-07-04'),
(5, 'O-', 'Al Rajhi Takaful', 'Ahmed Alshammari',
'+966555555555', '2026-07-05');
```

INSERT INTO doctor

```
(doctor_id, license_number, specialty, room_number,
consultation_fee, hire_date)
```

VALUES

```
(6, 'SCFHS-10001', 'General Medicine', 'A101', 150.00, '2021-02-01'),
(7, 'SCFHS-10002', 'Cardiology', 'A102', 300.00, '2020-09-15'),
(8, 'SCFHS-10003', 'Dermatology', 'B201', 250.00, '2022-01-10'),
(9, 'SCFHS-10004', 'Pediatrics', 'B202', 200.00, '2019-06-20'),
(10, 'SCFHS-10005', 'Dentistry', 'C301', 220.00, '2023-03-05');
```

INSERT INTO appointment

```
(appointment_id, patient_id, doctor_id, scheduled_at,
appointment_duration_minutes, status, reason)
```

VALUES

```
(1001, 1, 6, '2026-07-15 09:00:00', 30, 'Completed', 'Facial pain
and persistent nasal congestion'),
(1002, 2, 7, '2026-07-16 10:00:00', 45, 'Completed', 'Blood
pressure follow-up and medication review'),
(1003, 3, 8, '2026-07-17 11:00:00', 30, 'Completed', 'Itchy skin
rash on both hands'),
(1004, 5, 9, '2026-07-18 13:00:00', 30, 'Completed', 'Fever,
cough, and fatigue'),
(1005, 4, 10, '2026-07-19 15:00:00', 60, 'Completed', 'Tooth
sensitivity and localized pain');
```

INSERT INTO treatment

```
(treatment_id, appointment_id, treatment_date, diagnosis,
treatment_notes, treatment_cost)
```

VALUES

```
(2001, 1001, '2026-07-15', 'Acute bacterial sinusitis',
'Oral antibiotic course, analgesic as needed, hydration, and
follow-up if symptoms persist.', 120.00),
(2002, 1002, '2026-07-16', 'Essential hypertension - controlled',
```

```

        'Continue antihypertensive therapy, reduce sodium intake, and
        record home blood pressure readings.', 150.00),
        (2003, 1003, '2026-07-17', 'Mild contact dermatitis',
        'Use topical corticosteroid for seven days and avoid the
        suspected irritant.', 100.00),
        (2004, 1004, '2026-07-18', 'Seasonal influenza',
        'Supportive care, adequate fluids, rest, temperature monitoring,
        and return if breathing worsens.', 90.00),
        (2005, 1005, '2026-07-19', 'Early dental caries',
        'Cavity restored, fluoride applied, and oral hygiene instructions
        provided.', 250.00);

```

```

INSERT INTO medicine
    (medicine_id, medicine_name, strength, dosage_form, unit_price,
    stock_quantity, prescription_required)

```

VALUES

```

    (3001, 'Paracetamol',          '500 mg', 'Tablet',  0.50, 500,
    FALSE),
    (3002, 'Amoxicillin',          '500 mg', 'Capsule', 1.20, 200,
    TRUE),
    (3003, 'Amlodipine',          '5 mg',   'Tablet',  0.85, 180,
    TRUE),
    (3004, 'Hydrocortisone',       '1%',     'Cream',   18.00,  60,
    TRUE),
    (3005, 'Sodium Fluoride Gel',  '1.23%',  'Gel',     25.00,  40,
    TRUE);

```

```

INSERT INTO treatment_medicine
    (treatment_id, medicine_id, dosage, frequency, duration_days,
    quantity, instructions)

```

VALUES

```

    (2001, 3002, '500 mg',          'Three times daily', 7,  21,
    'Take after meals and complete the full course.'),
    (2001, 3001, '500 mg',          'Every 8 hours as needed', 5,
    15, 'Do not exceed the recommended daily dose.'),
    (2002, 3003, '5 mg',            'Once daily', 30, 30, 'Take at
    the same time each day.'),
    (2003, 3004, 'Thin topical layer', 'Twice daily', 7,  1, 'Apply
    only to the affected area.'),
    (2004, 3001, '500 mg',          'Every 8 hours as needed', 3,
    9, 'Use for fever or discomfort.'),
    (2005, 3005, 'Professional dose', 'Single application', 1,  1,
    'Applied by the dentist during the visit.'),
    (2005, 3001, '500 mg',          'Every 8 hours as needed', 2,
    6, 'Use only if post-treatment discomfort occurs.');
```

```

INSERT INTO payment

```

```

    (payment_id, appointment_id, amount, payment_date, payment_method,
    payment_status, reference_no)

```

VALUES

```

        (4001, 1001, 270.00, '2026-07-15 09:50:00', 'Card',
'Paid', 'PAY-2026-0001'),
        (4002, 1002, 450.00, '2026-07-16 11:00:00', 'Insurance',
'Paid', 'PAY-2026-0002'),
        (4003, 1003, 350.00, '2026-07-17 11:45:00', 'Card',
'Paid', 'PAY-2026-0003'),
        (4004, 1004, 290.00, '2026-07-18 13:40:00', 'Insurance',
'Paid', 'PAY-2026-0004'),
        (4005, 1005, 470.00, '2026-07-19 16:20:00', 'Bank Transfer',
'Paid', 'PAY-2026-0005');

COMMIT;

-----
-- TASK 2 VERIFICATION QUERIES
-- These basic table displays are provided for population screenshots.
-----

SELECT * FROM person ORDER BY person_id;
SELECT * FROM patient ORDER BY patient_id;
SELECT * FROM doctor ORDER BY doctor_id;
SELECT * FROM appointment ORDER BY appointment_id;
SELECT * FROM treatment ORDER BY treatment_id;
SELECT * FROM medicine ORDER BY medicine_id;
SELECT * FROM treatment_medicine ORDER BY treatment_id, medicine_id;
SELECT * FROM payment ORDER BY payment_id;

--
=====
-- TASK 3 - SQL OPERATIONS
-- Every assessed operation includes a short purpose statement.
UPDATE, DELETE,
-- and trigger demonstrations use ROLLBACK so the original academic
dataset is
-- preserved after the evidence queries are executed.
--
=====
-----
-- TASK 3.1 - SELECT STATEMENTS
-- Purpose: List the completed clinic appointments in chronological
order so the
-- clinic can review the encounter schedule and the reason for each
visit.
-----

```

```

SELECT
    appointment_id,
    scheduled_at,
    appointment_duration_minutes,
    status,
    reason
FROM appointment
WHERE status = 'Completed'
ORDER BY scheduled_at;

```

```

-- Purpose: Identify medicines with 100 units or fewer so clinic staff
can
-- prioritize stock monitoring and replenishment.

```

```

SELECT
    medicine_id,
    medicine_name,
    strength,
    stock_quantity,
    unit_price
FROM medicine
WHERE stock_quantity <= 100
ORDER BY stock_quantity ASC, medicine_name ASC;

```

```

-----
-- TASK 3.2 - JOIN QUERIES
-- Purpose: Combine appointments with the related patient, doctor, and
specialty
-- details to produce a readable appointment summary.
-----

```

```

SELECT
    a.appointment_id,
    a.scheduled_at,
    patient_person.full_name AS patient_name,
    doctor_person.full_name AS doctor_name,
    d.specialty,
    a.status
FROM appointment AS a
JOIN patient AS pat
    ON pat.patient_id = a.patient_id
JOIN person AS patient_person
    ON patient_person.person_id = pat.patient_id
JOIN doctor AS d
    ON d.doctor_id = a.doctor_id
JOIN person AS doctor_person
    ON doctor_person.person_id = d.doctor_id
ORDER BY a.scheduled_at;

```

```

-- Purpose: Connect each patient to the diagnosis and prescribed
medicines,

```

```
-- including dosage and treatment duration, for a complete
prescription report.
```

```
SELECT
    patient_person.full_name AS patient_name,
    t.diagnosis,
    m.medicine_name,
    m.strength,
    tm.dosage,
    tm.frequency,
    tm.duration_days
FROM patient AS pat
JOIN person AS patient_person
    ON patient_person.person_id = pat.patient_id
JOIN appointment AS a
    ON a.patient_id = pat.patient_id
JOIN treatment AS t
    ON t.appointment_id = a.appointment_id
JOIN treatment_medicine AS tm
    ON tm.treatment_id = t.treatment_id
JOIN medicine AS m
    ON m.medicine_id = tm.medicine_id
ORDER BY patient_person.full_name, m.medicine_name;
```

```
-- -----
-- TASK 3.3 - NESTED QUERY
-- Purpose: Find patients whose total paid amount is greater than the
average
-- paid transaction amount, helping the clinic identify higher-value
encounters.
-- -----
```

```
SELECT
    pat.patient_id,
    patient_person.full_name AS patient_name,
    SUM(pay.amount) AS total_paid
FROM patient AS pat
JOIN person AS patient_person
    ON patient_person.person_id = pat.patient_id
JOIN appointment AS a
    ON a.patient_id = pat.patient_id
JOIN payment AS pay
    ON pay.appointment_id = a.appointment_id
WHERE pay.payment_status = 'Paid'
GROUP BY pat.patient_id, patient_person.full_name
HAVING SUM(pay.amount) > (
    SELECT AVG(paid_payment.amount)
    FROM payment AS paid_payment
    WHERE paid_payment.payment_status = 'Paid'
)
ORDER BY total_paid DESC;
```

```

-----
-- TASK 3.4 - AGGREGATE FUNCTIONS WITH GROUP BY
-- Purpose: Summarize each doctor's workload and related financial
-- results using
-- appointment counts, average duration, treatment cost, and collected
-- payments.
-----

SELECT
    d.doctor_id,
    doctor_person.full_name AS doctor_name,
    d.specialty,
    COUNT(DISTINCT a.appointment_id) AS appointment_count,
    ROUND(AVG(a.appointment_duration_minutes), 2) AS
average_duration_minutes,
    SUM(t.treatment_cost) AS total_treatment_cost,
    SUM(CASE WHEN pay.payment_status = 'Paid' THEN pay.amount ELSE 0
END) AS total_collected
FROM doctor AS d
JOIN person AS doctor_person
    ON doctor_person.person_id = d.doctor_id
LEFT JOIN appointment AS a
    ON a.doctor_id = d.doctor_id
LEFT JOIN treatment AS t
    ON t.appointment_id = a.appointment_id
LEFT JOIN payment AS pay
    ON pay.appointment_id = a.appointment_id
GROUP BY d.doctor_id, doctor_person.full_name, d.specialty
ORDER BY appointment_count DESC, doctor_name;

-----

-- TASK 3.5 - UPDATE STATEMENT
-- Purpose: Demonstrate a five-percent price adjustment for
-- Hydrocortisone. The
-- transaction is rolled back after displaying the before-and-after
-- values.
-----

START TRANSACTION;

SET @original_hydrocortisone_price = (
    SELECT unit_price
    FROM medicine
    WHERE medicine_id = 3004
);

UPDATE medicine
SET unit_price = ROUND(unit_price * 1.05, 2)

```

```
WHERE medicine_id = 3004;

SELECT
    medicine_id,
    medicine_name,
    @original_hydrocortisone_price AS price_before_update,
    unit_price AS price_after_update
FROM medicine
WHERE medicine_id = 3004;

ROLLBACK;

-- -----
-- TASK 3.6 - DELETE STATEMENT
-- Purpose: Demonstrate removing one optional treatment-medicine
-- detail record.
-- ROLLBACK restores the deleted row so later queries keep the
-- original dataset.
-- -----

START TRANSACTION;

SELECT
    treatment_id,
    medicine_id,
    dosage,
    quantity
FROM treatment_medicine
WHERE treatment_id = 2005
    AND medicine_id = 3001;

DELETE FROM treatment_medicine
WHERE treatment_id = 2005
    AND medicine_id = 3001;

SELECT ROW_COUNT() AS deleted_rows;

SELECT COUNT(*) AS matching_rows_after_delete
FROM treatment_medicine
WHERE treatment_id = 2005
    AND medicine_id = 3001;

ROLLBACK;

SELECT COUNT(*) AS matching_rows_after_rollback
FROM treatment_medicine
WHERE treatment_id = 2005
    AND medicine_id = 3001;
```

```

-----
-- TASK 3.7 - VIEW
-- Purpose: Provide a reusable appointment summary that combines
patient,
-- doctor, treatment, and paid-payment information for reporting.
-----

DROP VIEW IF EXISTS vw_appointment_summary;

CREATE VIEW vw_appointment_summary AS
SELECT
    a.appointment_id,
    a.scheduled_at,
    patient_person.full_name AS patient_name,
    doctor_person.full_name AS doctor_name,
    d.specialty,
    a.status,
    t.diagnosis,
    t.treatment_cost,
    COALESCE(SUM(CASE WHEN pay.payment_status = 'Paid' THEN pay.amount
ELSE 0 END), 0.00)
        AS paid_amount
FROM appointment AS a
JOIN patient AS pat
    ON pat.patient_id = a.patient_id
JOIN person AS patient_person
    ON patient_person.person_id = pat.patient_id
JOIN doctor AS d
    ON d.doctor_id = a.doctor_id
JOIN person AS doctor_person
    ON doctor_person.person_id = d.doctor_id
LEFT JOIN treatment AS t
    ON t.appointment_id = a.appointment_id
LEFT JOIN payment AS pay
    ON pay.appointment_id = a.appointment_id
GROUP BY
    a.appointment_id,
    a.scheduled_at,
    patient_person.full_name,
    doctor_person.full_name,
    d.specialty,
    a.status,
    t.diagnosis,
    t.treatment_cost;

SELECT *
FROM vw_appointment_summary
ORDER BY scheduled_at;

```



```

-----
-----
-- TASK 3.8 - TRIGGER
-- Purpose: Protect medicine inventory by rejecting an excessive
prescription
-- quantity and automatically reducing stock when a new detail row is
inserted.
-----
-----
DROP TRIGGER IF EXISTS trg_treatment_medicine_stock;

DELIMITER $$

CREATE TRIGGER trg_treatment_medicine_stock
BEFORE INSERT ON treatment_medicine
FOR EACH ROW
BEGIN
    DECLARE v_available_stock INT DEFAULT -1;

    SELECT COALESCE(MAX(stock_quantity), -1)
    INTO v_available_stock
    FROM medicine
    WHERE medicine_id = NEW.medicine_id;

    IF v_available_stock = -1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'The selected medicine does not
exist.';
    ELSEIF NEW.quantity > v_available_stock THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Insufficient medicine stock for this
treatment.';
    ELSE
        UPDATE medicine
        SET stock_quantity = stock_quantity - NEW.quantity
        WHERE medicine_id = NEW.medicine_id;
    END IF;
END$$

DELIMITER ;

-- Trigger test: stock decreases from 40 to 38 after inserting two
units. The
-- transaction is rolled back afterward so the permanent stock returns
to 40.
START TRANSACTION;

SELECT
    medicine_id,
    medicine_name,
    stock_quantity AS stock_before_trigger

```

```
FROM medicine
WHERE medicine_id = 3005;

INSERT INTO treatment_medicine
(treatment_id, medicine_id, dosage, frequency, duration_days,
quantity, instructions)
VALUES
(2004, 3005, 'Professional dose', 'Single application', 1, 2,
'Temporary row used to verify the stock-control trigger.');
```

```
SELECT
    medicine_id,
    medicine_name,
    stock_quantity AS stock_after_trigger
FROM medicine
WHERE medicine_id = 3005;

ROLLBACK;

SELECT
    medicine_id,
    medicine_name,
    stock_quantity AS stock_after_rollback
FROM medicine
WHERE medicine_id = 3005;

SHOW TRIGGERS LIKE 'treatment_medicine';
```

8. Appendix C - Screenshot Evidence Index

This index maps every embedded MySQL Workbench screenshot to the Task 2 or Task 3 requirement it supports.

Screenshot	Evidence Purpose	Source File
2.1	PERSON table	09-1000380028.jpg
2.2	PATIENT table	03-1000380014.jpg
2.3	DOCTOR table	04-1000380018.jpg
2.4	APPOINTMENT table	10-1000380030.jpg
2.5	TREATMENT table	05-1000380020.jpg
2.6	MEDICINE table	01-1000380032.jpg
2.7	TREATMENT_MEDICINE table	02-1000380034.jpg
2.8	PAYMENT table	03-1000380036.jpg
3.1A	Completed appointments SELECT	04-1000380040.jpg
3.1B	Low-stock medicines SELECT	task3_01_select_low_stock.jpg
3.2A	Appointment details JOIN	task3_02_join_appointments.jpg
3.2B	Patient prescription JOIN	06-1000380072.jpg
3.3	Nested query	05-1000380070.jpg
3.4	Aggregate functions with GROUP BY	07-1000380024.jpg
3.5	UPDATE before/after	task3_05_update_result.jpg
3.6	DELETE and rollback	01-1000380161.jpg
3.7A	VIEW existence	01-1000380170.jpg
3.7B	VIEW full result	02-1000380171.jpg
3.8A	TRIGGER definition	03-1000380172.jpg
3.8B	TRIGGER stock test and rollback	04-1000380173.jpg